docs/01-factibilidad/fase4.md

Implementacion y Pruebas

1.Implementación realizada

Se utilizó PostgreSQL para motor de bases de datos, ademas de una API desarrollada con FastAPI y una interfaz web de apoyo para las consultas y reservas de asientos

La base de datos contiene las entidades necesarias para representar películas, cines, salas, asientos, funciones, usuarios, órdenes, pagos, boletos y bitácoras. Las relaciones se protegen con llaves primarias y foráneas, y las reglas que dependen del estado de los datos se controlan mediante restricciones, funciones y triggers.

La API tiene operaciones de POST, GET, DELETE entre otras. Se puede consultar acerca de disponibilidad de asientos, cines, funciones, salas, ordenes,pagos simulados, etc.Las relaciones estan protegido por PK´s y FK´S. Ademas los pagos son simulados

2. Método de prueba

Las pruebas se realizaron contra la base de datos PostgreSQL local y la API ejecutándose en el equipo de desarrollo. Para comprobar los flujos se usaron Swagger, consultas SQL en DBeaver y una prueba automatizada en Python. Se dividen asi: 1- comprobacion de funcionamiento y conexion de API (debe conectarse con el motor de base de datos) 2- luego se hacen pruebas funcionales 3- pruebas de integridad: comprobar que llaves foraneas y todas las restricciones impuestas 4- luego se realizan pruebas de concurrencia que verifican que no haya doble venta en un mismo boleto.

3: Diferentes casos de prueba

Caso	Procedimiento	Resultado observado	Estado
Conexion con BD	Consultar el endpoint de salud de la API	esto verificó que estaba conectada al motor	Aprobado
consultas de peliculas y funciones	registrar peliculas, horarios, funciones desde configuracion de API y frontend	Se recibieron respuestas exitosas y funciones se ven en app	Aprobado para pruebas funcionales
Consulta de disponibilidad	consultar asientos de funcion	Se recibieron los asientos y su estado. Todo esto incluye bloqueado y disponible en estado	Aprobado
Registro e inicio de sesion	registrar cuenta de prueba y autenticarla	registro respondió correctamente y Swagger permitió autenticar la cuenta	Aprobado
Reserva de asiento	Enviar una solicitud autenticada para reservar un asiento	Una solicitud recibió UTTP 201	Aprobado para el caso de prueba
Concurrencia sobre mismo asiento	Enviar 30 solicitudes simultáneas para reservar mismo asiento	Se dio respuesta `201` y 29 resultados `409 Conflict`	Aprobado para este caso
Liberacion de reserva	Liberar un asiento reservado y volver a tener su disponibilidad	El asiento dejó de aparecer bloqueado	Aprobado para caso ejecutado
Compra y generacion de boleto	Completar una compra simulada para fines académicos	Se genera boleto y el QR random respectivo	Aprobado para cada caso

4: Alcance

La pruebas muestran que los flujos principales del backend y frontend funcionan con datos de prueba utilizados y que los contextos de concurrencia si se resuelven. En una ocasion se aceptó la solicitud de asiento disputado y rechazó las siguinetes 29, lo que demuestra que no hay reservación doble en un mismo asiento. Se edito el valor de bloqueo_minutos de 10 a 2 minutos según los requisitos planteados

El objetivo de 50 000 TPS se aborda mediante una arquitectura escalable con balanceo de carga, múltiples instancias de la API, caché para lecturas apropiadas, agrupación de conexiones y PostgreSQL dimensionado para la carga. La integridad de las reservas se mantiene en la base de datos mediante transacciones y restricciones. En esta entrega se comprobó la exclusión de reservas con una prueba concurrente de 30 solicitudes; el objetivo de 50 000 TPS queda como requisito de rendimiento pendiente de validar mediante una prueba de carga en infraestructura dimensionada para ese volumen.

5: Conclusiones

Toda la implementacion permitió que el motor de base de datos (PostgreSQL), la API y la interfaz (frontend) pueden abarcar y solucionar bien los flujos principales de la app. Si se genera bloque temporal al seleccionar un asiento en una función Los resultados corresponden a pruebas locales con datos de prueba. La validación de la duración exacta del bloqueo debe coincidir con el requisito del proyecto, y la capacidad para soportar cargas de escala mundial queda fuera de lo demostrado por estas pruebas.

docs/03-diseno/desarrollo_diseno_conceptual_y_logico.md

Diseño Conceptual de la Base de Datos

Proyecto: Licitación para el Ecosistema Digital de Cines IMAX en todo el mundo **Fase:** 3, Diseño de la Base de Datos (nivel conceptual)

El diseño conceptual describe **qué información maneja el negocio y cómo se relaciona**, sin comprometerse con tipos de dato, llaves ni tablas físicas. Por eso aquí las relaciones de muchos a muchos aparecen tal como son en el negocio. El diseño lógico (`der.md`) las convierte en tablas con llaves, y el físico (`01_schema.sql`) las implementa en PostgreSQL.

---

1. Detalles antes de diagrama conceptual

Antes de generar esta parte, se deben tomar en cuenta algunos detalles como

Detalle	Explicación
Cardinalidad entre usuario y orden	1:N. Un usuario puede tener varias órdenes, pero cada orden pertenece a un solo usuario cliente
Ventas de taquilla y cortesías	Cuando el cliente no tiene cuenta (taquilla o cortesía) se usa un usuario genérico con el rol `cliente_mostrador`, de modo que `id_usuario` nunca es nulo. La relación de vendedor (`vendedor_asociado`) registra quién hizo la venta de taquilla o quién otorgó la cortesía
`franja_horaria` como entidad propia	La tarifa no es solo una N:M entre zona y tipo de entrada: depende del país, la zona, el tipo de entrada, el día de la semana y la franja horaria. La franja es un catálogo, así el horario de cada franja se define una sola vez
Unificación de nombres	Se usa `estado_asiento_funcion` (no `asiento_funcion`) y `vendedor_asociado` (no `id_usuario_vendedor`) en todos los documentos
Llaves foráneas compuestas en `boleto`	Son dos: (a) `(id_orden, id_funcion)` hacia `orden`, que asegura que el boleto sea de la función de su compra; (b) `(id_funcion, id_asiento)` hacia `estado_asiento_funcion`, que asegura que el asiento fue bloqueado antes en esa función
`ciudad` como entidad	Se relaciona con `pais` y con `cine`. Evita repetir texto de ubicación y separa ciudades con el mismo nombre en países distintos
`franja_horaria` y `configuracion`	`franja_horaria` es un catálogo del que depende `tarifa`. `configuracion` es la única entidad sin relaciones: guarda los parámetros globales del sistema
Reembolso con boleto opcional	Un pago aprobado puede llegar después de que expiró el bloqueo y el asiento ya se vendió a otro. Hay que devolver el dinero, pero no existe boleto
Regla entre asiento y sala	Un asiento seleccionado en una función debe pertenecer a la sala donde se proyecta esa función. Lo valida un trigger o procedimiento almacenado

2.Resumen de entidades del proyecto

Situacion	Entidades seleccionadas en el contexto
Ubicación y estructura	`pais,ciudad,cine,sala,zona,asiento,dispositivo_puerta,configuracion`
cartelera y programacion	`pelicula,genero,pelicula_genero,funcion,franja_horaria`
precios y promociones	`tipo_entrada,tarifa,promocion`
privacidad, seguridad y labores(accesos)	`rol,modulo,rol_permiso,usuario`
transacciones y ventas	`orden,pago,metodo_pago,boleto,reembolso`
Bitácoras para auditoria	`transaccion_log,asiento_auditoria_log,acceso_puerta_log,admin_operacion_audit`

3.Diagrama Entidad-Relación

3.1: Diagrama de 26 tablas de negocio erDiagram rol ||--o{ usuario : "tiene" rol ||--o{ rol_permiso : "recibe" modulo ||--o{ rol_permiso : "se asigna en"

pais ||--o{ ciudad : "tiene" ciudad ||--o{ cine : "aloja" cine ||--o{ sala : "contiene" sala ||--o{ asiento : "tiene" zona ||--o{ asiento : "clasifica" sala ||--o{ dispositivo_puerta : "tiene"

pelicula ||--o{ pelicula_genero : "pertenece" genero ||--o{ pelicula_genero : "agrupa" pelicula ||--o{ funcion : "se proyecta en" sala ||--o{ funcion : "aloja"

pais ||--o{ tarifa : "define moneda y precio" zona ||--o{ tarifa : "define precio" tipo_entrada ||--o{ tarifa : "define precio" franja_horaria ||--o{ tarifa : "define precio"

usuario ||--o{ orden : "compra" usuario |o--o{ orden : "vende en taquilla" funcion ||--o{ orden : "es comprada en" promocion |o--o{ orden : "se aplica a" orden ||--o{ pago : "se paga con" metodo_pago ||--o{ pago : "usa"

funcion ||--o{ estado_asiento_funcion : "tiene estado de" asiento ||--o{ estado_asiento_funcion : "tiene estado en" usuario |o--o{ estado_asiento_funcion : "bloquea"

orden ||--o{ boleto : "genera" estado_asiento_funcion ||--o{ boleto : "se vende como" tipo_entrada ||--o{ boleto : "clasifica"

pago ||--o{ reembolso : "devuelve" boleto |o--o{ reembolso : "origina" usuario |o--o{ reembolso : "autoriza"

rol { smallint id_rol PK varchar nombre_rol UK varchar descripcion } modulo { smallint id_modulo PK varchar nombre UK } rol_permiso { smallint id_rol PK, FK smallint id_modulo PK, FK varchar nivel "ninguno, lectura, ejecucion, escritura, total" } usuario { bigint id_usuario PK smallint id_rol FK varchar nombre varchar correo UK varchar telefono varchar contrasena_hash boolean activo timestamptz fecha_registro } pais { smallint id_pais PK varchar nombre UK varchar moneda } ciudad { int id_ciudad PK smallint id_pais FK varchar nombre } cine { int id_cine PK int id_ciudad FK varchar nombre varchar direccion varchar zona_horaria } sala { int id_sala PK int id_cine FK varchar nombre_sala int capacidad_total boolean sala_esta_activa } zona { smallint id_zona PK varchar nombre UK "platea, palco, VIP, general" } asiento { int id_asiento PK int id_sala FK smallint id_zona FK varchar fila smallint numero } dispositivo_puerta { int id_dispositivo PK int id_sala FK varchar nombre } genero { smallint id_genero PK varchar nombre UK } pelicula { int id_pelicula PK varchar titulo text sinopsis varchar clasificacion_edad smallint duracion_minutos varchar poster_url boolean pelicula_esta_activa } pelicula_genero { int id_pelicula PK, FK smallint id_genero PK, FK } funcion { bigint id_funcion PK int id_pelicula FK int id_sala FK timestamptz fecha_hora_inicio timestamptz fecha_hora_fin varchar estado_funcion "programada, cancelada, finalizada" } franja_horaria { smallint id_franja PK varchar nombre time hora_inicio time hora_fin } tipo_entrada { smallint id_tipo_entrada PK varchar nombre UK "adulto, nino, tercera_edad, estudiante" boolean requiere_credencial } tarifa { int id_tarifa PK smallint id_pais FK smallint id_zona FK smallint id_tipo_entrada FK smallint id_franja FK smallint dia_semana "0 a 6, obligatorio" numeric precio_base } promocion { int id_promocion PK varchar nombre_promocion varchar tipo_regla "2x1, descuento_dia" numeric descuento_pct int condicion_minima smallint dia_semana "obligatorio si descuento_dia; vacio si 2x1" boolean activa } metodo_pago { smallint id_metodo_pago PK varchar nombre UK "tarjeta, paypal, billetera, transferencia, efectivo, cortesia" } orden { bigint id_orden PK bigint id_usuario FK "cliente; obligatorio" bigint vendedor_asociado FK "taquilla o administrativo; opcional" bigint id_funcion FK int id_promocion FK varchar canal "web, app, taquilla, administrativo" varchar estado_orden "pendiente, pagada, fallida, anulada" numeric subtotal

numeric descuento numeric cargo_servicio numeric total varchar moneda timestamptz fecha_creacion } pago { bigint id_pago PK bigint id_orden FK smallint id_metodo_pago FK varchar estado_pago "pendiente, aprobado, rechazado" numeric monto varchar token_pasarela "nulo con efectivo y cortesia" char ultimos4 "nulo con efectivo y cortesia" timestamptz fecha_operacion } estado_asiento_funcion { bigint id_funcion PK, FK int id_asiento PK, FK varchar estado "disponible, bloqueado, ocupado" bigint id_usuario FK "quien bloquea; nulo si disponible" timestamptz expiracion_bloqueo "nulo si no esta bloqueado" } boleto { bigint id_boleto PK bigint id_orden FK "FK compuesta (id_orden, id_funcion)" bigint id_funcion FK " FK compuesta a estado_asiento_funcion" int id_asiento FK "FK compuesta a estado_asiento_funcion" smallint id_tipo_entrada FK numeric precio_final uuid codigo_qr UK varchar estado_boleto "emitido, usado, anulado" varchar origen "online, taquilla, cortesia" timestamptz fecha_emision timestamptz fecha_uso } reembolso { bigint id_reembolso PK bigint id_pago FK bigint id_boleto FK "nulo si el pago fue tardio" bigint id_usuario_autoriza FK numeric monto_reembolso varchar motivo boolean forzado timestamptz fecha_reembolso } configuracion { smallint id_configuracion PK smallint bloqueo_minutos smallint max_asientos_en_orden smallint tolerancia_antes_min smallint tolerancia_despues_min smallint reembolso_horas_antes numeric cargo_servicio_porcentaje }

3.2: Diagrama de 4 tablas de auditoría

- Recordar que estas no tienen FK fisicas.Solo guardan identificadores como referencias lógicas.

erDiagram transaccion_log { bigint id_log PK timestamptz fecha_hora PK "llave de particion" bigint id_orden bigint id_pago varchar evento "intento, confirmacion, emision, rechazo" numeric monto numeric impuesto varchar moneda varchar ref_pasarela jsonb detalle } asiento_auditoria_log { bigint id_log PK timestamptz fecha_hora PK "llave de particion" bigint id_funcion int id_asiento bigint id_usuario varchar evento "bloqueo, expiracion, liberacion, venta" timestamptz expira_en } acceso_puerta_log { bigint id_log PK timestamptz fecha_hora PK "llave de particion" bigint id_boleto int id_dispositivo varchar resultado "exitoso, duplicado, rechazado_horario, rechazado_credencial, funcion_incorrecta, invalido" varchar motivo } admin_operacion_audit { bigint id_log PK timestamptz fecha_hora PK "llave de particion" bigint id_usuario smallint id_rol inet ip_origen varchar operacion "cortesia, cambio_asiento, reembolso_forzado, config" varchar entidad bigint id_entidad jsonb valor_anterior jsonb valor_nuevo }

4.Entidades

Entidad	Descripción
`pais`, `ciudad`	Ubicación geográfica de cines. Recordar que cada cine pertenece a una ciduad
`cine`	sucursal IMAX ubicada en una ciudad específica
`sala`	Sala de proyección de un cine
`zona`	puede ser plate,palco,VIP o general (para el asiento)
`asiento`	asiento físico de una sala (tiene fila, numero y zona)
`dispositivo_puerta`	lector de qr en la entrada de sala
`configuracion`	relacionado a parametros como moneda y demas (parámetros globales sistema)
`pelicula`	catálogo de películas. tiene titulo,sinopsis,clasificación, duración y póster
`genero`	género de la película
`pelicula_genero`	relacion N:M entre película y genero
`funcion`	incluye pelicula,sala,hora de inicio, hora de fin y estado
`franja_horaria`	catálogo de horarios

Entidad	Descripción
tipo_entrada	especifica si es para niño,tercera edad,adulto,etc
tarifa	precio base basado en función del país,zona y demas detalles
promocion	promocion aplicable como descuentos,2x1.
rol	rol en el sistema de un usuario(cliente, taquillero,operador,contador,cliente_mostrador,administrador)
modulo	modulo funcional del sistema
rol_permiso	matriz de privilegios de cada rol sobre el módulo
usuario	usuario del sistema
orden	compra. Tiene su cliente,funcion,montos,canal y puede tener vendedor asociado
pago	intento de pago de una orden; con token y últimos 4 dígitos
metodo_pago	tarjeta,efectivo,cortesía,PayPal,etc
boleto	boleto digital emitido tras pago aprobado. Tiene un QR único
reembolso	para devolver dinero
estado_asiento_funcion	si esta ocupado, disponible o bloqueado
transaccion_log	auditoría de pagos, confirmaciones y emisión de boletos
asiento_auditoria_log	auditoría de bloqueos,liberaciones y ventas de asientos
acceso_puerta_log	registro de cada escaneo de QR en puerta
admin_operacion_audit	auditoria de operaciones manuales del administrador

5. Relaciones y cardinalidades

Relación	Cardinalidad	Comentario
pais → ciudad	1 : N	
ciudad → cine	1 : N	
cine → sala	1 : N	
sala → asiento	1 : N	El mapa de asientos de una sala
zona → asiento	1 : N	Platea, palco, VIP, general
sala → dispositivo_puerta	1 : N	
rol → usuario	1 : N	Un usuario tiene un solo rol

Relación	Cardinalidad	Comentario
rol ↔ modulo	N : M (vía rol_permiso)	Matriz de privilegio mínimo
pelicula ↔ genero	N : M (vía pelicula_genero)	
pelicula → funcion	1 : N	
sala → funcion	1 : N	Sin traslapes
pais, zona, tipo_entrada, franja_horaria → tarifa	1 : N cada una	Una tarifa combina las cuatro, con día de la semana y precio
funcion ↔ asiento	N : M (vía estado_asiento_funcion)	Estado de cada asiento en cada función
usuario → estado_asiento_funcion	0..1 : N	Quién tiene el bloqueo
usuario → orden (cliente)	1 : N	Obligatorio; en taquilla se usa `cliente_mostrador`
usuario → orden (vendedor)	0..1 : N	Solo canal taquilla o administrativo
funcion → orden	1 : N	
promocion → orden	0..1 : N	Una orden puede no tener promoción
orden → pago	1 : N	Varios intentos de pago posibles
metodo_pago → pago	1 : N	
orden → boleto	1 : N	De uno a seis
estado_asiento_funcion → boleto	1 : 0..1 vigente	A lo sumo un boleto no anulado
tipo_entrada → boleto	1 : N	
pago → reembolso	1 : N	
boleto → reembolso	0..1 : N	Sin boleto cuando el pago llegó tarde
usuario → reembolso (autoriza)	0..1 : N	Sin usuario si la devolución es automática

docs/03-diseno/entidades_archivo_final.md

Identificación de Entidades y Atributos

Proyecto: Licitación para el Ecosistema Digital de Cines IMAX **Fase:** 3, Diseño de la Base de Datos

Este documento identifica las entidades del sistema a partir de las reglas de negocio: para cada regla se determina qué información hay que guardar y en qué entidades. Los nombres de tablas y columnas son los mismos que se usan en `der.md`, donde se presentan el diagrama entidad-relación, las cardinalidades y las restricciones.

---

1. Criterios generales: para tomarlos como base y reglas a utilizar

- **Cartelera no es una entidad.** Es una vista (consulta) que combina `funcion`, `pelicula`, `sala` y `cine`, filtrando funciones activas y futuras (RN-01).
- **Convención de nombres:** tablas en singular y `snake_case`. Las llaves foráneas se escriben `id_<tabla>`.
- **Fechas y horas:** siempre `timestamptz`, porque los cines están en distintos países.
- **Seguridad:** nunca se guarda el número de tarjeta ni el CVV; solo `token_pasarela` y los últimos 4 dígitos (RN-09). Las contraseñas se guardan como hash.
- **Dependencias:** una entidad dependiente lleva FK a la entidad de la que depende (función → película y sala, asiento → sala, boleto → orden).

---

2. De las reglas de negocio a las entidades

2.1 Catálogo y cartelera (RN-01, RN-02)

Qué se necesita guardar	Entidades	Atributos
Detalle de cada película: título, sinopsis, clasificación de edad, duración, póster y si está activa	`pelicula`	`id_pelicula` [PK], `titulo`, `sinopsis`, `clasificacion_edad`, `duracion_minutos`, `poster_url`, `pelicula_esta_activa`
Género(s) de cada película. Una película puede tener varios, por eso no es una entidad simple	`genero`, `pelicula_genero`	`genero`: `id_genero` [PK], `nombre`. `pelicula_genero`: `id_pelicula` [PK, FK], `id_genero` [PK, FK]
Ubicación de los cines en el mundo (para filtrar por ubicación)	`cine`	`id_cine` [PK], `id_ciudad` [FK], `nombre`, `direccion`, `zona_horaria`
Salas de cada cine	`sala`	`id_sala` [PK], `id_cine` [FK], `nombre_sala`, `capacidad_total`, `sala_esta_activa`
Programación: qué película se proyecta, en qué sala y cuándo	`funcion`	`id_funcion` [PK], `id_pelicula` [FK], `id_sala` [FK], `fecha_hora_inicio`, `fecha_hora_fin`, `estado_funcion`
Detalles acerca del pais especifico	`pais`	`id_pais [PK]`, `nombre`, `moneda`
Detalles acerca de las ciudades específicas	`ciudad`	`id_ciudad[PK]`,`id_pais[FK]`,`nombre`

Notas:

- La ubicación exacta de la sala (RN-02) se obtiene por `sala` → `cine`, por lo que no se repite en `sala`.
- `fecha_hora_fin` permite evitar traslapes en una misma sala y calcular la ventana de tolerancia de RN-16.
- `estado_funcion` solo admite un valor a la vez: `programada`, `cancelada` o `finalizada` (restricción de constraint `CHECK`).

2.2 Asientos y mapa de la sala (RN-03, RN-04, RN-05)

Qué se necesita guardar	Entidades	Atributos
Zonas de la sala (platea, palco, VIP, general)	`zona`	`id_zona` [PK], `nombre`
Distribución física de cada sala: es decir los asientos	`asiento`	`id_asiento` [PK], `id_sala` [FK], `id_zona` [FK], `fila`, `numero`. Restricción `UNIQUE (id_sala, fila, numero)`, es decir que la combinacion de esos 3 atributos, no se puede repetir.
Estado de cada asiento en cada función, incluido el bloqueo temporal de 2 minutos	`estado_asiento_funcion`	`id_funcion` [PK, FK], `id_asiento` [PK, FK], `estado`, `id_usuario` [FK, nulo], `expiracion_bloqueo` (`timestamptz`, nulo)

Notas:

- `estado_asiento_funcion.estado` admite `disponible`, `bloqueado` u `ocupado`. El estado "reservado" del enunciado corresponde a `bloqueado`: una reserva temporal mientras el usuario completa el pago.
- La PK compuesta `(id_funcion, id_asiento)` impide físicamente que un mismo asiento aparezca dos veces en una función.
- La fila se crea **bajo demanda**, cuando el asiento se bloquea por primera vez. La ausencia de fila equivale a "disponible".
- `expiracion_bloqueo` se fija en el momento del bloqueo más 2 minutos, e `id_usuario` identifica quién tiene el bloqueo.
- El límite de asientos por transacción (RN-04, máximo 6) no es una entidad: se valida en el procedimiento de reserva.
- el asiento de un `estado_asiento_funcion` debe pertenecer a la sala de su funcion (validado por trigger o procedimiento)

2.3 Tarifas y promociones (RN-06, RN-07, RN-08)

Qué se necesita guardar	Entidades	Atributos
Tipos de entrada	`tipo_entrada`	`id_tipo_entrada` [PK], `nombre` (adulto, niño, tercera edad, estudiante), `requiere_credencial`
Precio según zona del asiento, tipo de entrada y día de la semana	`tarifa`	`id_tarifa` [PK],`id_pais[FK]`, `id_zona` [FK], `id_franja[FK]`,`id_tipo_entrada` [FK], `dia_semana` (0 a 6 y obligatorio), `precio_base`
Promociones y sus condiciones	`promocion`	`id_promocion` [PK], `nombre_promocion`, `tipo_regla` (2x1, descuento por día), `descuento_pct`, `condicion_minima`, `dia_semana`, `activa`
los distintos horarios o franjas	`franja_horaria`	`id_franja`[PK],`nombre`,`hora_inicio`,`hora_fin`

Notas:

- Se va a crear un catálogo de franjas horarias al que apunta `tarifa`, esto permite que el horario de cada franja o "espacio" (es lo mismo) se defina una única vez y un cambio no va a obligar a modificar varias tarifas
- también debemos tomar en cuenta otro punto: el solapamiento. Con el catalogo de franjas, las mismas viven en una tabla pequeña y asi nunca hay solapamiento, pues se controla desde esa tabla. NOTA: EN NIVEL LOGICO LAS FRANJAS HORARIAS NO PUEDEN SOLAPARSE (podria usarse trigger)
- la franja de zona horaria se compara con la hora de inicio de la funcion específica
- para temas de duplicados: la regla que evita que 2 precios para la misma combinación de zona,tipo, dia y franja es una de UNIQUE
- existe otro problema. ahorita `dia_semana` significa todos los dias, por lo tanto debemos poner una regla NOT NULL y el "todos los dias se va a expresar con siete filas (una por dia), del 0 al 6

-única sobre (id_pais, id_zona, id_tipo_entrada, dia_semana, id_franja). promocion: dia_semana opcional, obligatorio si tipo_regla es descuento_dia y vacío si es 2x1

- `tipo_entrada` es un catálogo con `UNIQUE` en `nombre`, de modo que solo existen los perfiles definidos.
- `promocion` **no lleva FK hacia `tarifa`**, porque el precio final se calcula combinando ambas. Lo que sí se guarda es qué promoción se aplicó en cada compra (campo `id_promocion` en `orden`).
- `requiere_credencial` indica si en la puerta hay que validar un documento de identidad o credencial (RN-06).

2.4 Usuarios, roles y permisos (RN-17)

Qué se necesita guardar	Entidades	Atributos
Usuarios del sistema (clientes y personal)	`usuario`	`id_usuario` [PK], `id_rol` [FK], `nombre`, `correo` (`UNIQUE`), `telefono`, `contrasena_hash`, `activo`, `fecha_registro`
Roles	`rol`	`id_rol` [PK], `nombre_rol`, `descripcion`
Módulos del sistema	`modulo`	`id_modulo` [PK], `nombre`
Qué puede hacer cada rol en cada módulo (matriz de privilegio mínimo)	`rol_permiso`	`id_rol` [PK, FK], `id_modulo` [PK, FK], `nivel` (ninguno, lectura, ejecucion, escritura, total)

Nota: los 6 roles son cliente, taquillero, operador de sala, contador,cliente_mostrador(sin filas en rol_permiso, el usuario genérico tendrá ese rol, un correo interno fijo y activo=verdadero. Ademas ese genérico tendrá un contraseña_hash que ninguna contraseña produce) y administrador/gerente. `rol_permiso` guarda la matriz del enunciado como datos, de modo que el RBAC pueda consultarse y demostrarse.

2.5 Compra y pago (RN-08, RN-09, RN-10)

Qué se necesita guardar	Entidades	Atributos
Compra completa: quién, para qué función, montos y estado	`orden`	`id_orden` [PK] , `id_usuario` [FK], `id_funcion` [FK] , `id_promocion` [FK, nulo], `canal` (web, app, taquilla,administrativo), `estado_orden` (pendiente, pagada, fallida, anulada), `subtotal`, `descuento`, `cargo_servicio`, `total`, `fecha_creacion`,`vendedor_asociado`[FK a `usuario`,nulo],`moneda`

Qué se necesita guardar	Entidades	Atributos
Intentos de pago y respuesta de la pasarela	`pago`	`id_pago` [PK], `id_orden` [FK], `id_metodo_pago` [FK], `estado_pago` (aprobado, rechazado, pendiente), `monto`, `token_pasarela[nulo]`, `ultimos4[nulo]`, `fecha_operacion`
Métodos de pago (para el reporte de métodos preferidos)	`metodo_pago`	`id_metodo_pago` [PK], `nombre` (efectivo,cortesia,tarjeta, PayPal, billetera, transferencia)

Notas: -`vendedor_asociado[FK,nulo]`

- `monto` puede ser 0 solo con metodo `cortesía`, y token y últimos 4 quedan vacíos con efectivo y cortesía
- (id_orden, id_funcion) deben ser unicas porque asi referencian ambas columnas juntas hacia orden y tambien es restricción
- `orden` y `pago` están separadas porque una compra puede tener varios intentos de pago (uno rechazado y otro aprobado).
- `subtotal`, `descuento` y `total` se guardan como una fotografía financiera: no deben cambiar si luego cambia una tarifa.
- Si el pago es rechazado, la orden pasa a `fallida` y los asientos se liberan de inmediato (RN-10).
- (id_orden,id_funcion) es FK compuesta en boleto
- (id_funcion,id_asiento) es FK compuesta en boleto

2.6 Emisión del boleto (RN-11, RN-12, RN-13)

Qué se necesita guardar	Entidades	Atributos
Boleto digital único con su código QR, creado solo después de un pago aprobado	`boleto`	`id_boleto` [PK], `id_orden` [FK], `id_funcion` [FK], `id_asiento` [FK], `id_tipo_entrada` [FK], `precio_final`, `codigo_qr` (`UUID`, `UNIQUE`, `NOT NULL`), `estado_boleto` (emitido, usado, anulado), `origen` (online, taquilla, cortesia), `fecha_emision`, `fecha_uso`

Notas:

- Un boleto **no existe** hasta que el pago es aprobado. Por eso no hay estado "pendiente" ni boletos sin QR: la condición de RN-11 se cumple por construcción.
- `(id_funcion, id_asiento)` es una **FK compuesta** hacia `estado_asiento_funcion`: un boleto solo puede existir sobre un asiento que fue bloqueado antes por el flujo de compra.
- `id_tipo_entrada` permite validar la credencial en la puerta (RN-06), y `precio_final` conserva lo realmente cobrado.
- Como red de seguridad contra la doble venta, se agrega un índice único parcial: `UNIQUE (id_funcion, id_asiento) WHERE estado_boleto <> 'anulado'`.
- Un boleto anulado libera el asiento para una nueva venta.

RESUMEN DE 2.5 y 2.6

- **FK compuesta 1:** `(id_orden, id_funcion)` → `orden`. Garantiza que el boleto sea de la misma función que su orden.

- **FK compuesta 2:** `(id_funcion, id_asiento)` → `estado_asiento_funcion`. Garantiza que el asiento fue bloqueado antes en esa función.
- Se necesitan 2 fk compuestas, para verificar casos: la fk compuesta 1 (id_orden,id_funcion) revisa si existe una orden con tal funcion seleccionada y si coincide con lo del boleto, se acepta. La fk compuesta 2 dice que si existe un asiento en x funcion seleccionada dentro de estado_asiento_funcion y coincide con lo del boleto, pues se acepta.
- en el contexto que termina una función, lo que se hace es purgar los datos que se bloquearon y se abandonaron(que no tenian boleto), y ademas se mantiene FK compuesta 2, porque eso va a garantizar que todo boleto nace de que un usuario lo bloquea y se van a purgar solo filas(registros) de funciones finalizadas que no tienen boleto. No se puede hacer cascada porque borraría todo el historial de datos de venta (boletos vendidos), con los otros no hay problema porque no se generaron ventas ni afectaciones ahi
- los registros con su boleto se mantienen en la BD, y con las estimaciones realizadas en el der(que pueden ser aprox unos 15 millones de boletos al mes y 60 bytes por fila) lo cual puede ser como 1GB al mes, como 60 GB a 5 años.Es llevadero para lo dado en el contexto
- Tomemos en cuenta un caso límite: que pasa si un cliente obtiene un reembolso. En ese caso entonces vamos a purgar filas de funciones finalizadas sin ningún boleto (ni activo ni anulado).En este caso borra filas de estado_asiento_funcion y solo en funciones finalizadas que no tienen un boleto asociado; es decir que los asientos con boleto activo no se borra, los asientos con boleto anulado tampoco, y un asiento bloqueado(osea que el usuario lo reservó, pero nunca siguió con el pago) y abandonado si se purga.

-Básicamente los boletos no se borran para que sirvan como registros fiscales

-`orden`: el canal pasa a web,app,taquilla, administrativo

Problema 4 a resolver Se crea un catálogo de franjas horaria al que apunta tarifa, asi el horario de cada franja se define una sola vez y un cambio no obliga a modificar varias tarifas.

- También debemos registrar quien hizo la venta para un cliente, para eso vamos a crear una columna extra en `orden` como `vendedor_asociado` y de por medio en `usuario` haremos un usuario generico. Luego dejamos activo=verdadero y que el usuario genérico tenga un correo interno fijo y un contrasena_hash que ninguna contraseña produce, así no puede iniciar sesión

En eso tenemos una limitación, la cual es que un solo registro con un id_usuario se comparte por todos los clientes de mostrador. Luego hacemos un marcador en este modelo: agregar un sexto rol `cliente_mostrador` en tabla `rol`, sin filas en `rol_permiso`

- un cliente recibe una entrada en el mostrador (para los genéricos), ya sea físico o solo con qr (pero ambos tienen ese codigo). RN-13 solo aplica a ventas online, y la venta de taquilla es una excepción declarada.Lo que la FK garantiza es que, si hay vendedor, exista como usuario. Se localiza boleto por `codigo_qr` y el reembolso se va a registrar con `id_usuario_autoriza`
- Regla: si canal es taquilla o administrativo, vendedor_asociado es obligatorio; en web o app debe estar vacío.
- `vendedor_asociado`: Es opcional porque las ventas online no tienen vendedor. La FK garantiza que, si hay vendedor, exista como usuario
- estado_asiento_funcion.id_usuario guarda al vendedor solo en ventas presenciales
- reembolso sigue aplicando. Sigue la regla de 2 horas (RN-19) y que quien tenga el QR puede solicitarlo
- Los reportes de clientes excluyen al usuario genérico
- pero deberiamos hacer un trigger para tabla `orden`, pues la fk solo comprueba q usuario exista, no su rol. Ese trigger se ejecuta aunque alguien inserte la orden sin pasar por procedimiento de venta.
- agregamos efectivo en `metodo_pago`. Ahi en ese caso `token_pasarela` y `ultimos4` en `pago` deben ser opcionales y ahi pago rechazado por pasarela no aplica

- `cliente_mostrador` no se puede asignar como rol a un usuario real porque solo puede existir un usuario con ese rol y ademas una orden con canal web o app no puede usar a ese usuario

2.7 Control de acceso en puerta (RN-14, RN-15, RN-16)

Qué se necesita guardar	Entidades	Atributos
Dispositivos de escaneo (tótems y lectores)	`dispositivo_puerta`	`id_dispositivo` [PK], `id_sala` [FK], `nombre`
Cada intento de escaneo, aunque sea rechazado	`acceso_puerta_log`	`id_log` [PK], `fecha_hora` [PK, llave de partición], `id_boleto`, `id_dispositivo`, `resultado` (exitoso, duplicado, rechazado_horario,rechazado_credencial, funcion_incorrecta, invalido), `motivo`

2.8 Reembolsos (RN-19)

Qué se necesita guardar	Entidades	Atributos
Devolución de dinero y anulación del boleto	`reembolso`	`id_reembolso` [PK], `id_boleto` [FK,nulo] (es nulo cuando un pago aprobado llega despues de que el bloqueo expiró y el asiento ya se vendió a otro: hay que devolver el dinero, pero no existe boleto), `id_pago` [FK], `id_usuario` [FK], `monto_reembolso`, `motivo`, `forzado`, `fecha_reembolso`

Nota: la política de "hasta 2 horas antes de la función" se valida en el procedimiento de reembolso, no en una entidad. Si el reembolso es forzado, además se registra en `admin_operacion_audit`.

2.9 Auditoría a 5 años (RN-20)

El enunciado exige **cuatro tablas separadas**, con estos nombres, particionadas por fecha. No llevan FK físicas (son masivas y de solo inserción); guardan los identificadores como referencias lógicas.

Tabla	Qué registra	Atributos
`transaccion_log`	Cada intento de pago, confirmación de pasarela, montos, impuestos y emisión de boletos	`id_log` [PK], `fecha_hora` [PK], `id_orden`, `id_pago`, `evento`, `monto`, `impuesto`, `moneda`, `ref_pasarela`, `detalle` (jsonb)
`asiento_auditoria_log`	Bloqueo, liberación por expiración o venta de un asiento	`id_log` [PK], `fecha_hora` [PK], `id_funcion`, `id_asiento`, `id_usuario`, `evento`, `expira_en`
`acceso_puerta_log`	Cada escaneo de QR (ver sección 2.7)	Ver sección 2.7
`admin_operacion_audit`	Toda modificación manual de un administrador: cortesías, cambios de asiento, reembolsos forzados y cambios de configuración	`id_log` [PK], `fecha_hora` [PK], `id_usuario`, `id_rol`, `ip_origen`, `operacion`, `entidad`, `id_entidad`, `valor_anterior` (jsonb), `valor_nuevo` (jsonb)

2.10 Restricción de salas con ventas activas (RN-18)

No requiere una entidad nueva. Se implementa con un **trigger** sobre `sala` y `asiento` que rechaza eliminar o modificar la distribución si existen boletos vendidos o reservas para funciones futuras de esa sala.

2.11 Parámetros del sistema (RN-04, RN-05, RN-08, RN-16, RN-17, RN-19)

Qué se necesita guardar	Entidades	Atributos
Valores de las reglas de negocio que IMAX puede querer cambiar sin modificar código	`configuracion`	`id_configuracion` [PK], `bloqueo_minutos`, `max_asientos_en_orden`, `tolerancia_antes_min`, `tolerancia_despues_min`, `reembolso_horas_antes`, `cargo_servicio_porcentaje`

Parámetros, valores iniciales y rangos permitidos:

Parámetro	Valor inicial	Rango permitido	Regla
`bloqueo_minutos`	2	1 a 15	RN-05
`max_asientos_en_orden`	6	1 a 10	RN-04
`tolerancia_antes_min`	30	0 a 120	RN-16
`tolerancia_despues_min`	15	0 a 60	RN-16
`reembolso_horas_antes`	2	0 a 48	RN-19
`cargo_servicio_porcentaje`	por definir	0 a 100	RN-08

Notas:

- **Una sola fila.** La tabla guarda un único conjunto de valores. Regla: no puede existir una segunda fila, por eso `id_configuracion` solo admite un valor.
- **Una columna por parámetro.** Cada columna tiene su propio tipo y su propio rango de validación, algo que no se podría hacer si todos los valores fueran texto en una tabla de clave-valor.
- **Sin modificar código.** Los procedimientos leen estos valores en lugar de tenerlos escritos. Cambiar un parámetro es actualizar la fila.
- **Quién puede cambiarlos.** Solo el Administrador/Gerente, según la matriz de roles (RN-17: "configuración de reglas y precios").
- **Auditoría.** Cada cambio se registra en `admin_operacion_audit` con la operación `config`, la entidad `configuracion`, el valor anterior, el valor nuevo, el usuario, su rol y su IP (RN-20).
- **Los bloqueos ya activos no cambian.** Si `bloqueo_minutos` pasa de 2 a 5, los bloqueos vigentes conservan su hora de vencimiento original. El cambio afecta solo a los bloqueos nuevos.
- **Cargo por servicio como porcentaje.** Se guarda como porcentaje y no como monto, porque un monto dependería de la moneda de cada país.
- **Limitaciones:** los valores son globales (no varían por país ni por cine), y agregar un parámetro nuevo exige modificar la tabla.

3. Cobertura de reglas de negocio

Regla	Cubierta por
RN-01	`funcion`, `pelicula` (vista de cartelera)
RN-02	`pelicula`, `funcion`, `sala`, `cine`
RN-03	`asiento`, `estado_asiento_funcion`
RN-04	Validación en el procedimiento de reserva,`configuracion.max_asientos_en_orden`
RN-05	`estado_asiento_funcion.expiracion_bloqueo`,`configuracion.bloqueo_minutos` y `asiento_auditoria_log`
RN-06	`tipo_entrada.requiere_credencial`, `boleto.id_tipo_entrada`,`acceso_puerta_log`
RN-07	`promocion`, `orden.id_promocion`
RN-08	`tarifa`,`franja_horaria`,`pais`, `orden` (subtotal, descuento, cargo, total),`configuracion.cargo_servicio_porcentaje`
RN-09	`pago` (solo token y últimos 4 dígitos)
RN-10	`pago.estado_pago`, `orden.estado_orden`
RN-11	`boleto` solo nace de orden con su pago verificado y aprobado, incluidas cortesías (lo cual implica monto=0)
RN-12	`boleto.codigo_qr` único
RN-13	`boleto` consultado desde el perfil del usuario cliente (solo aplica a ventas online)
RN-14	`boleto.estado_boleto`, `acceso_puerta_log`
RN-15	`acceso_puerta_log`, `funcion`
RN-16	`funcion.fecha_hora_inicio` y ventana de tolerancia en el procedimiento de check-in,`configuracion`
RN-17	`rol`(el cual incluye `cliente_mostrador`), `modulo`, `rol_permiso`, `usuario`,`configuracion`
RN-18	Trigger sobre `sala` y `asiento`
RN-19	`reembolso`,`configuracion.reembolso_horas_antes`
RN-20	Las 4 tablas de la sección 2.9 incluida la `cortesia`

Todas las reglas de negocio (RN-01 a RN-20) quedan cubiertas.

4. Lista final de entidades

`pais`, `ciudad`, `cine`, `sala`, `zona`, `asiento`, `dispositivo_puerta`, `configuracion`, `pelicula`, `genero`, `pelicula_genero`, `funcion`, `franja_horaria`, `tipo_entrada`, `tarifa`, `promocion`, `rol`, `modulo`, `rol_permiso`, `usuario`, `orden`, `pago`, `metodo_pago`, `boleto`, `reembolso`, `estado_asiento_funcion`, `transaccion_log`, `asiento_auditoria_log`, `acceso_puerta_log` y `admin_operacion_audit`.

Son 30 tablas en total (26 de negocio y 4 de auditoría)

Dónde	Qué escribir
2.1 pais	id_pais [PK], nombre, moneda
2.1	ciudad (nueva) id_ciudad [PK], id_pais [FK], nombre
2.1	cine id_cine [PK], id_ciudad [FK], nombre, direccion, zona_horaria (sin pais ni ciudad como texto)
2.3	franja_horaria (nueva) id_franja [PK], nombre, hora_inicio, hora_fin. Regla: las franjas no se solapan
2.3	tarifa id_tarifa [PK], id_pais [FK], id_zona [FK], id_tipo_entrada [FK], id_franja [FK], dia_semana (obligatorio, 0 a 6), precio_base. Única sobre (id_pais, id_zona, id_tipo_entrada, dia_semana, id_franja)
2.3	promocion dia_semana opcional. Regla: obligatorio si tipo_regla es descuento_dia; vacío si es 2x1
2.5	orden agrega vendedor_asociado [FK a usuario, opcional] y moneda
2.5	metodo_pago tarjeta, paypal, billetera, transferencia, efectivo, cortesia
2.5	pago token_pasarela y ultimos4 opcionales. monto puede ser 0 solo con método cortesia
2.7	acceso_puerta_log agrega rechazado_credencial en resultado
2.11	configuracion (nueva) bloqueo_minutos, max_asientos_en_orden, tolerancia_antes_min, tolerancia_despues_min, reembolso_horas_antes, cargo_servicio_porcentaje, cada uno con su rango. Regla: una sola fila

---

5. Decisión de diseño: estado del asiento por función

El estado de cada asiento en cada función se modela con la tabla `estado_asiento_funcion`, de PK compuesta `(id_funcion, id_asiento)`, creada bajo demanda.

Criterio	Justificación
Unicidad anti doble venta	La PK compuesta impide físicamente dos filas para el mismo asiento en la misma función
Reserva atómica	Una única sentencia `INSERT ... ON CONFLICT DO UPDATE ... WHERE` concede el bloqueo a una sola solicitud; las demás obtienen 0 filas afectadas
Mapa de asientos	Una consulta sobre `asiento` con un `LEFT JOIN` a `estado_asiento_funcion` para la función elegida
Volumen	Solo se almacenan los asientos bloqueados o vendidos (unos 20 millones de filas al mes), no todos los asientos de todas las funciones (unos 67 millones al mes)
Orden bloqueo → venta	`boleto` referencia a `estado_asiento_funcion` con una FK compuesta

La alternativa considerada fue modelar los bloqueos en una tabla `bloqueo_asiento` separada de `boleto`, con índices únicos parciales en ambas. Se descartó porque obliga a consultar y mantener dos tablas para conocer el estado de un asiento, y porque la unicidad se apoya en índices parciales en lugar de una llave primaria. El detalle de la sentencia de bloqueo está en `der.md`.

Problemas identificados con diseño presentado

1.moneda y zona horaria

- como IMAX tiene varios cines en el mundo tenemos que generalizar la moneda para `precio_base`

Por ende, vamos a guardar la moneda en una tabla pais

- la zona horaria se guarda en `cine`, pues un pais puede tener varias zonas horarias y con la zona por cien, cada cine ubica hora local.
- `ciudad` es una tabla con `id_ciudad`,`id_pais[FK]`,`nombre`. `cine` apunta a ciudad.Asi logramos que el nombre de un país no se repite en cada cine
- La tarifa se relaciona a un país con un `id_pais`. Y aplica lo de temas de duplicados: la regla que evita que 2 precios para la misma combinación de zona,tipo, dia y franja es una de UNIQUE.
- el dinero de cada compra: `orden` va a guardar la moneda. `pago`,`boleto` y `reembolso` se basan en la moneda de su orden específica
- el dia y franja se calculan con hora local del cine se hace con `fecha_hora_inicio`

DETALLES A TOMAR EN CUENTA: se usarán reportes por moneda

- ninguna FK impide que una funcion use tarifa de otro país. El procedimiento de compra válida que la tarifa pertenezca al país de la función, cruzando `funcion->sala->cine->pais`
- asumimos solo una moneda por pais (no se modelan paises con varias monedas). En el caso de `impuesto` en `transaccion_log` , su valor se registra por transacción.

Otros pendientes

#1: Parametros configurables. En las RN o casos de prueba vienen ejemplos aleatorios, pero IMAX puede desear otros. Se presenta una solucion para eso

Crear una tabla `configuracion` con una sola fila y una columna por parámetro Sus columnas son: `bloqueo_minutos` (entre 1 y 15 mins, con propuesta de 2 mins),`max_asientos_en_orden` (entre 1 y 10),`tolerancia_antes_min` (entre 0 y 120),`tolerancia_despues_min` (entre 0 y 60), `reembolso_horas_antes` (entre 0 y 48), `cargo_servicio_porcentaje`(entre 0 y 100).

- cada columna tiene su tipo y su rango de validación. Cambiarlos no requieren modificar codigo y cada cambio ya queda auditado. Limitaciones: valores son globales, no por país y agregar un parámetro nuevo exige modificar la tabla.

#2: cortesías sin pago Decisión: una cortesía es una orden de total 0, con un pago de monto 0 y método `cortesia`, aprobado. Así RN-11 se cumple sin excepciones, `boleto.id_orden` sigue siendo obligatorio y las dos FK compuestas de A1 funcionan igual.

Quién la otorga: solo el Administrador/Gerente. Su id queda en `vendedor_asociado`, con canal `administrativo`.

A quién se asigna: al usuario del invitado si tiene cuenta. Si no, al usuario genérico cliente_mostrador.

Reglas: si canal es taquilla o administrativo, vendedor_asociado es obligatorio; en web o app, debe estar vacío. El trigger valida rol taquillero para taquilla y rol administrador para administrativo. El usuario genérico no puede usarse en web o app. Si boleto.origen es cortesia, el pago de su orden usa el método cortesia, y al revés.

Efectos: pago.monto admite 0 solo con método cortesia. RN-09 y RN-10 no aplican, no hay pasarela. El reporte de métodos preferidos excluye las cortesías. Cuentan para el máximo de asientos (RN-04). Cada cortesía registra una fila en admin_operacion_audit con operación cortesia y el id de la orden (RN-20).

#punto	Consecuencia	Decisión propuesta
1	El catálogo metodo_pago necesita un valor nuevo	Agregar cortesia (además de efectivo)
2	pago.monto hoy probablemente exige un valor positivo	Debe admitir 0, y el 0 solo vale si el método es cortesia
3	No hay pasarela de por medio	token_pasarela y ultimos4 ya son opcionales por el efectivo. RN-09 y RN-10 no aplican a este pago, igual que con el efectivo
4	El reporte de “métodos de pago preferidos”	Se calcula sin las cortesías, porque no es un método que elige el cliente. Anótalo, porque cambia el resultado de esa consulta
5	Redundancia entre boleto.origen = cortesia y el método de pago	Regla: si el origen es cortesia, el pago de su orden usa el método cortesia, y al revés
6	orden.id_usuario es obligatorio	se asigna al usuario del invitado, o al genérico si no tiene cuenta
7	canal solo admite web, app y taquilla	se agrega el canal `administrativo`
8	RN-20	Cada cortesía registra una fila en admin_operacion_audit con la operación cortesia y el id de la orden
9	RN-04 (máximo 6 por orden)	La cortesía también cuenta para el límite
10	Quién puede otorgarla	Solo Administrador/Gerente, según la matriz de roles

#3: credencial en la puerta

- RN-06 dice que la validación “puede” hacerse físicamente en la puerta, y no exige dejar constancia de que se hizo. Por eso no se registra cada verificación exitosa. Sí se registra el rechazo, para tener trazabilidad de incidencias. El boleto solo se marca como usado cuando el ingreso se concede.
- en acceso_puerta_log.resultado se agrega rechazado_credencial, para dejar constancia de cuándo el guardia rechazó a alguien por no tener credencial.
- el boleto pasa a usado solo cuando el ingreso se concede. Si se rechaza por credencial, el boleto queda en emitido y el cliente puede volver con su documento.

#4: estado reservado Se mantienen 3 estados y además "reservado" es equivalente a `bloqueado`. Estado azul se deduce en consulta del mapa.Los asientos fuera de servicio no se van a modelar

- lo que quiere decir eso es que quien bloquea un asiento se pone en color azul el asiento

docs/03-diseno/entidades_borrador.md

Fase de identificación de entidades y atributos

--ENTIDADES--

- Funciones: cartelera como tal no es una entidad (es una vista, que presenta las funciones en imax cinema). Con atributos: id_funcion,horarios(podria ser compuesta, pero con un timestamp se llena completa),estado(solo uno posible)

Pelicula: con atributos de id_pelicula,titulo, sinopsis, clasificacion de edad, duracion y genero. PERO NOTE QUE PELICULA SI ES UNA ENTIDAD FUERTE, PERO LA FUNCION SE DEBE RELACIONAR CON LA PELICULA, PERO LA FUNCION DEPENDE DE QUE HAYA PELICULA, ENTONCES DEBE HABER UNA FK Sala: id_sala, num_sala, ubicacion_sala (LO MISMO ACÁ, SE NECESITARÁ UNA FK)

- RN-01 + RN-02> -> para cumplir esta regla necesito guardar: detalles relacionados a cada película que se reproduce (basado en películas activas y no se hayan reproducido). Entidad: pelicula (detallada anteriormente) con atributos: id_pelicula,titulo,sinopsis,clasificacion_edad,duracion,estado
- Basado en el contexto del negocio brindado, en el cual mencionan que tienen cines por todo el mundo, podemos agregar lo siguiente

Necesito la entidad sucursal_cine con atributos: id_sucursal,nombre_sucursal,ubicacion, pais,ciudad y ademas, la entidad sala con atributos: id_sala, nombre_sala, capacidad_total y una FK que apunta a cine_id, lo cual haría cine_id[FK]

Regla Base	Se necesita guardar	Entidades Necesarias
RN-01	programaciones de funciones relacionada con una película en x sala con su horario completo específico	funcion: con atributos: id_funcion,id_pelicula[FK] (porque una funcion depende de que haya pelicula),fecha_hora_inicio,estado_funcion
RN-03	conocer si un asientos está ocupado o no físicamente por cada sala y conocer su distribución	asiento, con atributos: id_asiento,id_sala[FK] (pues se necesita referenciar la sala para los asientos),fila,numero, zona
RN-03, RN-04 y RN-05	estado que se refleja temporalmente acerca de los asientos, en el momento que usuario interacciona con mapa en la app	asientos_en_funcion con atributos: id_funcion[PK,FK],id_asiento[PK,FK],estado, id_reservacion_temporal_usuario,expiracion_bloqueo (este debería ser un timestamp y lo vamos a fijar en 2 minutos)
RN-06,RN-07 y RN-08	información acerca de las tarifas por cada tipo de usuario, pues puede variar si son niños, estudiantes o tercera edad y todo lo relacionado a descuentos	tarifas con atributos: id_tarifa[PK],tipo_perfil (a este luego se le hará un constraint para verificar que solo este en uno de los posibles perfiles), precio_base. Además tenemos promociones con atributos: id_promocion[PK],nombre_promocion,tipo_regla, descuento_en_porcentaje,condicion_minima (no se debe incluir una FK acá, pues el precio final se calcula con la unión de ambas tablas)
RN-09 y RN-17	los intentos de pago relacionado con la respuesta de la pasarela y que no se almacene en la BD datos sensibles del usuario, ademas necesitamos tener una tabla específica para identificar a los distintos usuarios(cliente, operador de sala, etc) y otra aparte para el rol del usuario	usuarios con atributos:id_usuario[PK],nombre,correo,telefonos, id_rol[FK],contraseña_en_hash (porque si se pone un atributo de contraseña seria poco seguro en caso de ataques a la BD),fecha_registro. Otra entidad "operacion_pago" con atributos: id_transaccion[PK], id_usuario[FK],monto_total_a_pagar,metodo_de_pago,estado_pago(solo puede ser uno de 3 posibles, se hace un constraint para este atributo),token_de_pasarela (para cumplir con temas de seguridad),fecha_operacion.Y por último la entidad rol con atributos: id_rol[Pk], nombre_rol (para identificar si es operador de salas, algun cliente, taquilleros, etc), descripcion

Regla Base	Se necesita guardar	Entidades Necesarias
RN-11, RN-12 y RN-13	se necesita que se genere el boleto digital único con su debido código QR, solo cuando se haya efectuado y confirmado el pago	"boletos" con atributos: id_boleto,id_transaccion[FK],id_funcion[FK],id_asiento[FK], codigo_qr (este en sql debe ser unico), estado_boleto (emitido, utilizado, anulado. En caso de que no sea emitido, no se genera el qr)
RN-14,RN-15 y RN-16	se debe registrar los intentos de escaneo de cada qr para la entrada a la sala, pues un qr solo puede ser escaneado una unica ves	"acceso_puerta_con_qr_log" con atributos: id_log_acceso,id_boleto[FK],id_dispositivo_escaneador (pues hay varios dispositivos, entonces hay que identificarlos), horario_completo_de_escaneo (es un timestamp), resultado (ahi se verifica si el qr sirve o no: exitoso, duplicado, rechazado_por_horario)
RN-18	se necesita algun registro como una bitácora o agenda que registre modificaciones manuales	"auditoria_de_administracion" con atributos: id_auditoria[PK],id_usuario[FK] (para referencia a un administrador),accion_realizada (para saber si es un reembolso, alguna cortesía,etc), descripcion_del_cambio, ip_origen (direccion ip donde se dió esa acción), horario_de_cambio (es de timestamp).
RN-19	para registrar como tal un reembolso o la transacción relacionada a eso	"reembolso" con atributos: id_reembolso, id_boleto,id_transaccion, monto_de_reembolso, detalle_motivo, fecha_reembolso,id_usuario[FK]